HU-06 — PII reemplazada por tokens fijos antes de persistir

> **Como** Administrador de Base de Datos **quiero** que toda PII identificada sea reemplazada por tokens fijos previo a su almacenamiento o registro de logs **para** garantizar la irreversibilidad del proceso y evitar la fuga de datos personales sensibles. — Prioridad Alta · 5 SP

Estado anterior (revisión del 16-oct): **Parcial**. Lo que va al LLM, las métricas, los eventos de seguridad y los logs ya no tenían PII, pero `respuestas.justificacion_usuario`, los archivos de evidencia en disco y los textos ARCOP se guardaban en claro, y el mapa de entidades era reversible.

Criterios de aceptación

#	Criterio	Cómo se cumple	Prueba
C1	Sustitución por etiquetas estandarizadas (`[RUT_ANON]`, `[NOMBRE_ANON]`, `[CORREO_ANON]`, `[TEL_ANON]`)	Mismo motor (`NERSanitizer` v1.1.0) y mismas etiquetas fijas para lo que va al LLM **y** para lo que se guarda. También `[DIRECCION ANON]` y `[DATO_CODIFICADO_ANON]`	`test_hu06_anonimizacion.py::test_anonimizador_usa_etiquetas_fijas`
C2	Enmascaramiento en memoria previo a persistencia en BD o logs	La justificación y el nombre del archivo se anonimizan en memoria **antes** del `INSERT/UPDATE`; si el motor falla no se guarda nada (503 `ANONIMIZACION_NO_DISPONIBLE`). El archivo de evidencia nunca toca el disco en claro: se guarda cifrado y el worker lo reemplaza por su texto anonimizado. Los logs solo llevan ids, hashes y tipos de error	`test si falla la anonimizacion no se guarda_nada`, `test_worker_deja_solo_el_texto_anonimizado`
C3	Pruebas de regresión que verifican ausencia de texto plano	`test_hu06_regresion_pii.py` siembra PII por las 5 entradas del sistema y revisa **todas las tablas leídas en crudo**, los archivos en disco, los logs (todos los niveles) y lo enviado al LLM	`test_regresion_no_queda_pii_en_bd_disco_ni_logs`

Decisiones del PO (16-oct-2026)

1. Archivos de evidencia en disco → cifrar en reposo y luego dejar solo el texto anonimizado

Momento	Qué hay en disco
API recibe el archivo	Archivo **cifrado** (Fernet, subclave `evidencias` de `ENTITY_MAP_KEY`). Nombre en disco = UUID + extensión (el nombre original podía tener PII)
Worker valida	Lo descifra **en memoria**; la capa anti prompt-injection revisa el **original** (también los datos codificados) y la capa de IA lo anonimiza antes del LLM
Worker termina (validada, rechazada o error)	Se escribe **solo el texto anonimizado**, se actualiza `storage_path` y se **borra** el cifrado
Si el motor de anonimización falla	El archivo **queda cifrado** (nunca en claro) y se registra en el log, sin contenido

Por qué no "anonimizar al subir": la anonimización reemplaza los datos codificados (Base64/Hex) por `[DATO_CODIFICADO_ANON]`, y la detección de prompt injection necesita verlos. Cifrar primero mantiene intacta la capa de

seguridad del worker.

`evidencias.hash_archivo` sigue siendo el SHA-256 del archivo **original** (detección de duplicados): un hash de un documento completo no permite reconstruirlo.

2. Mapa de entidades → irreversible

- `sanitizaciones.mapa_cifrado` queda en `NULL`. Se conservan tipo, posición, fuente, **huella HMAC** y conteos por tipo (lo que necesitan la auditoría y las métricas de `/api/ia/metricas`).
- La huella permite responder un ARCOP de acceso ("¿en qué evidencias aparece mi RUT?") calculando la huella del dato que entrega el titular, sin guardar el dato.
- Interruptor: `GUARDAR_MAPA_ENTIDADES_CIFRADO=true` vuelve al mapa reversible. Si el PO lo activa, debe registrarse como excepción a este criterio.
- Nota de alcance: la huella es un seudónimo con clave. Quien tenga `ENTITY_MAP_KEY` podría probar RUTs uno por uno y encontrar coincidencias; para nombres y correos no es práctico. Por eso la clave vive solo en el `.env` del servidor.

3. Excepción ARCOP → textos cifrados, no etiquetados

Los textos de una solicitud ARCOP **se necesitan en claro para atender el derecho** (una rectificación trae el dato correcto; un acceso puede requerir identificar al titular), así que reemplazarlos por etiquetas haría imposible cumplir la Ley 21.719. Se documentan como excepción:

Dato	Tabla.columna	Tratamiento
Descripción de la solicitud (titular)	`solicitudes_arcop.descripcion`	Cifrado Fernet (subclave `arcop`)
Respuesta del responsable	`solicitudes_arcop.respuesta`	Cifrado
Comentarios del historial (titular y admin)	`eventos_solicitud_arcop.comentario`	Cifrado
Email del invitado	`usuario_temporal.email`	En claro: es la credencial de acceso del titular (se compara al iniciar sesión). Se borra al aprobar una supresión

Garantías de la excepción:

- **Solo se descifran en la API**, para el titular dueño de la solicitud y para `admin_therion` (control de acceso ya existente: un titular recibe 404 en solicitudes ajenas).
- La consulta pública por código (`/api/arcop/seguimiento/{codigo}`) no expone ninguno de estos campos.
- Nunca se escriben en logs. En la BD, en los respaldos (`pg_dump`) y para quien consulte PostgreSQL directamente son ilegibles (`enc:v1:...`).
- Una **supresión aprobada** los sigue reemplazando por `[anonimizado]` (se guarda también cifrado).
- Implementación: tipo de columna `TextoCifrado` (`app/core/cifrado_columnas.py`). La columna sigue siendo `TEXT`: **no hay migración de esquema**. Filas antiguas sin cifrar se leen igual y el script de migración las cifra.
- Sin `ENTITY_MAP_KEY` no se puede escribir (fail-closed).
- Pendiente para el PO (no bloquea la HU): plazo de **retención** de estos textos después de cerrar la solicitud (p. ej. anonimizarlos N meses después del cierre).

4. Prueba de regresión → `tests/test_hu06_regresion_pii.py`

Entrada sembrada con PII	Verificación
Justificación (`POST /api/respuestas`)	Se guarda con etiquetas; reenviarla igual no cuenta como cambio
Justificación con prompt injection	422; el extracto del evento de seguridad no tiene PII
Archivo de evidencia y su nombre (`contrato_12.345.678-5.txt`)	Nombre `contrato_[RUT_ANON].txt`; cifrado antes del worker; solo texto anonimizado después
Evidencia con prompt injection	Bloqueada sin llamar al LLM; extracto y archivo sin PII
Solicitud ARCOP completa (descripción, comentario, información, respuesta)	Cifrada en BD y legible por la API

Al final: volcado **en crudo** de todas las tablas del modelo, de la carpeta de evidencias, de los logs (nivel DEBUG, con tracebacks) y de stdout/stderr, buscando el valor original y variantes (`12345678` , `jperez@` , `8765 4321` , `Juan Pérez` …). Además: `mapa_cifrado` siempre NULL y nada de PII en lo que salió hacia el LLM.

Archivos

Tipo	Archivo	Qué hace
nuevo	`app/application/ai/anonimizador.py`	`AnonimizadorPII` : anonimiza textos y nombres de archivo antes de persistir (fail-closed)
nuevo	`app/core/cifrado_columnas.py`	Tipo de columna `TextoCifrado` (excepción ARCOP)
nuevo	`app/infrastructure/storage/archivo_cifrado_storage.py`	Storage de evidencias cifrado en reposo
nuevo	`scripts/migrar_pii_hu06.py`	Migra los datos guardados antes de HU-06 (simulación por defecto)
nuevo	`tests/test_hu06_anonimizacion.py`	17 pruebas unitarias (20 casos)
nuevo	`tests/test_hu06_regresion_pii.py`	Prueba de regresión C3
modificado	`app/infrastructure/security/cifrado.py`	Subclaves HKDF por propósito y `CifradorDatos` (textos y bytes)
modificado	`app/core/config.py`	`GUARDAR_MAPA_ENTIDADES_CIFRADO` (por defecto `false`)
modificado	`app/infrastructure/repositories/registro_ia_repository.py`	No guarda `mapa_cifrado` salvo que se active
modificado	`app/infrastructure/llm/factory.py`	`crear anonimizador()` y `crear_storage_evidencias()`
modificado	`app/infrastructure/storage/local_storage.py`	Nombre en disco = UUID (sin el nombre original)
modificado	`app/infrastructure/security/prompt_guard.py`	Extracto de eventos con el mismo motor NER (fallback a `seguridad.py`)
modificado	`app/application/use_cases/create_respuesta.py`	Anonimiza la justificación tras el guard y antes de guardar; error 503
modificado	`app/application/use_cases/upload_evidence.py`	Nombre de archivo anonimizado
modificado	`app/application/use_cases/process_evidence.py`	Reemplaza el archivo cifrado por el texto anonimizado
modificado	`app/api/routes/respuestas.py` · `evidencias.py` · `app/worker/tasks.py`	Inyectan anonimizador y storage cifrado
modificado	`app/domain/models/solicitud_arcop.py`	`descripcion` , `respuesta` y `comentario` con `TextoCifrado`
modificado	`app/domain/models/sanitizacion.py` · `app/application/ai/gateway.py`	Documentación
modificado	`tests/conftest.py`	Si spaCy no está instalado, las pruebas de API en proceso usan el motor con un NLP simulado (en Docker se usa spaCy real)
modificado	`tests/test_e2e_capa_ia.py` · `test_live_capa_ia.py`	Verifican huellas en vez de descifrar el mapa; e2e: con el NER caído la API ya no recibe la evidencia (503)
modificado	`tests/test_solicitudes_arcop.py` · `test hu02 matriz estados.py` · `test_seguridad_integracion.py`	Nombres en disco = UUID; storage cifrado
modificado	`.env.example` · `../README.md`	Variables y documentación

Sin migraciones Alembic nuevas (ningún cambio de esquema).

Verificación (entorno sin spaCy, NLP simulado): **357 aprobadas, 9 omitidas** (antes 336 + 9; +21 pruebas de HU-06). La prueba de regresión falla, como corresponde, si se desactiva cualquiera de los cuatro controles (anonimizar la justificación, cifrar la evidencia, no guardar el mapa, cifrar ARCOP). Falta correrla en Docker con spaCy real como evidencia de cierre.

Cómo aplicarlo

```
# 1. Copiar los archivos del zip sobre Proyecto/ y reconstruir
docker compose up -d --build

# 2. Pruebas
docker compose exec worker pytest tests -v -rxX
```

```
docker compose exec worker pytest tests/test_hu06_regresion_pii.py -v -s   # evidencia de C3

# 3. Datos existentes (respaldar antes: la anonimización no se deshace)
docker compose exec -T db pg_dump -U postgres complaince_db > respaldo_pre_hu06.sql
docker compose exec worker python scripts/migrar_pii_hu06.py             # simulación: muestra conteos
docker compose exec worker python scripts/migrar_pii_hu06.py --aplicar   # aplica
```

El script anonimiza justificaciones, nombres de archivo y extractos antiguos; reemplaza los archivos ya procesados por su texto anonimizado y cifra los pendientes; vacía `mapa_cifrado`; cifra los textos ARCOP. Es idempotente y no imprime contenido, solo conteos.

Efectos a tener en cuenta

- **La API ahora necesita el motor NER** para guardar respuestas con justificación y para subir evidencias (antes solo el worker). La imagen Docker ya trae spaCy y la API lo precarga al iniciar. Sin spaCy, `POST /api/respuestas` responde 503 en vez de guardar en claro. Costo: ~25 ms por justificación (p95 de T-04).
- `GET /api/respuestas/{id}` devuelve la justificación **anonimizada**: el usuario ve sus datos personales reemplazados por etiquetas. Es el comportamiento esperado de la HU.
- **Rotar `ENTITY_MAP_KEY`** deja ilegibles los textos ARCOP y las evidencias aún no procesadas cifrados con la clave anterior (la API muestra `[texto cifrado ilegible]`). Rotarla exige recifrar.
- Las evidencias binarias (PDF, DOCX) se siguen leyendo como texto UTF-8 (deuda #6 del README): el texto anonimizado que queda de ellas es tan legible como lo que hoy recibe el LLM.